



<Conceitos iniciais>

[UNIFEI | Universidade Federal de Itajubá](#)

Instituto de Ciências Tecnológicas

ECO – <Dev. Mobile>

eduardo.felipe@unifei.edu.br

Tipos de app

Nativo

Híbrido

Aplicação nativa

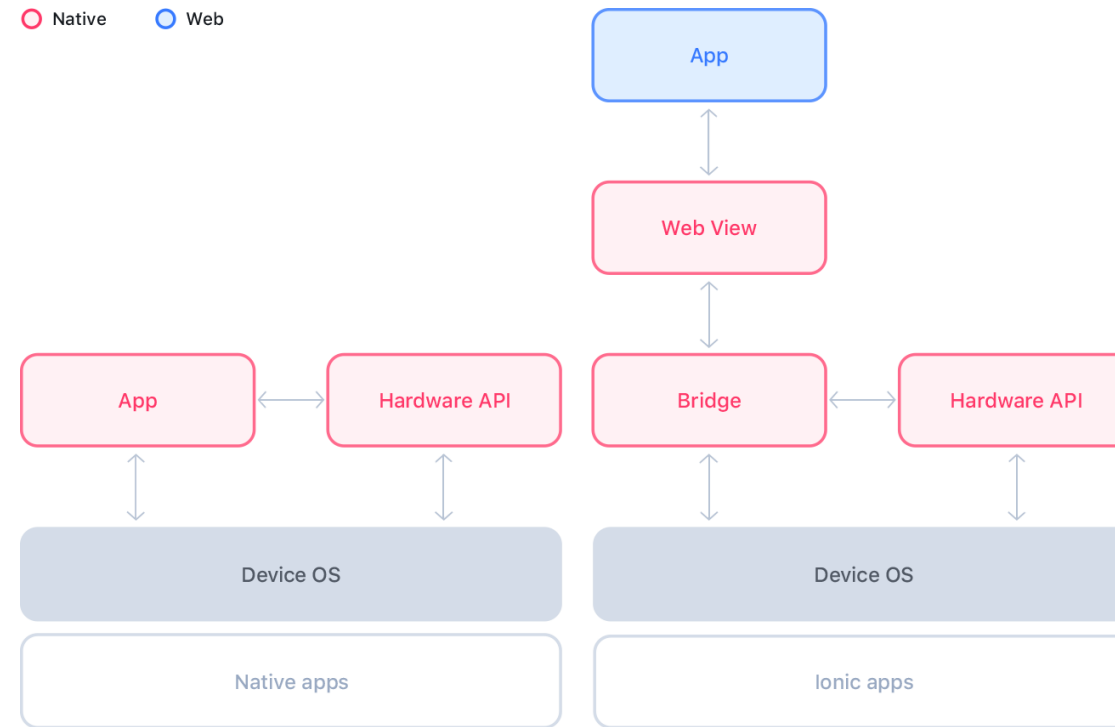
- São aplicações desenvolvidas e compiladas especificamente para um determinado Sistema Operacional.
- Android
 - Ambiente (IDE): Android Studio
 - Linguagens: Kotlin, Java.
- Mac (iOS, macOS)
 - Ambiente (IDE): Xcode
 - Linguagens: Swift, Objective C.
- Prós: Permite o acesso a todos os recursos/funcionalidades (APIs) do Sistema Operacional e possui melhor performance.
- Contras: Código sem aproveitamento para migrar para outro Sistema Operacional, retrabalho ao desenvolver para ambos ambientes.

Web app e PWA (progressive web apps)

- Web app
 - Um app que funciona “dentro” de um navegador (browser).
 - Como seu ambiente de execução é o navegador, pode funcionar no desktop e/ou mobile.
 - Pode confundir o usuário com a interface (botões, controles) do navegador.
- PWA
 - Um app que funciona “dentro” de um navegador (browser), **porém**, com a aparência de um app tradicional.
 - São instalados pelo site do proprietário, **não** podem ser publicados nas lojas (*stores*) dos sistemas nativos.
- Ambos formatos possuem grandes restrições de acessos às APIs nativas dos sistemas operacionais.

Híbrido - Ionic

- É um framework que permite desenvolver apps com base em tecnologias web (html, css, javascript).
- Também compartilha do conceito de componentes.



<https://ionicframework.com>

<https://ionicframework.com/docs/core-concepts/fundamentals>

.NET Multi-Platform App UI

Crie aplicativos móveis e desktop nativos e multiplataforma em uma única estrutura.

Comece a usar

Ler os documentos

React Native e Flutter

- Possui camadas de software (*bridge*) para “traduzir” o código escrito em linguagem “não-nativa” (javascript) para o código nativo de cada plataforma.
- Grande vantagem de escrever o app “uma vez” e ser portado para ambas plataformas. Desenvolvimento multiplataforma.
- Como ponto de atenção pode-se considerar que novas funcionalidades (APIs) podem demorar a ser implementadas. Visto que os fabricantes desenvolvem para suas APIs e ambientes *nativos*, restando à comunidade de desenvolvimento, codificar o acesso a estes novos recursos.

Multiplataforma (tendência)

- A famosa empresa de criação de ambientes de desenvolvimento (IDEs) [JetBrains](#) lança uma forma de desenvolver multiplataforma.
- Usando a IDE Android Studio e a linguagem Kotlin pode-se gerar aplicações para Android e iOS:
 - <https://www.jetbrains.com/compose-multiplatform/>
- Características:
 - Deve-se aprender a linguagem Kotlin (direcionada ao nativo Android)
 - Deve-se dominar a técnica de interface "compose", criada para o Android nativo
- Obs: Acesse a área educacional para solicitar licenças de IDEs gratuitas para o perfil acadêmico: <https://www.jetbrains.com/pt-br/community/education/#students>

Multiplataforma

- A concorrência...
- Mais um framework para multiplataforma, com grandes dependências de React:
 - <https://lynxjs.org>
 - <https://youtu.be/aqgQatFPNSw?si=biLhehjapSkssQAH>

Terminal e Git

Prompt de comando

- **Importante** ter conhecimento sobre operações via **prompt de comando** (terminal – PowerShell)
- Pelo menos saber comandos básicos sobre:
 - Criar pastas (diretório)
 - Entrar e sair de pastas
 - Listar conteúdo das pastas
- Estude:
 - <https://youtu.be/9JCElq7svL4>
 - <https://youtu.be/QZ2nyxzZXPY>

Desejável que o profissional TI iniciante conheça...

- Linha de **comandos** no Sistema Operacional
- Versionador de arquivos: GIT - <https://git-scm.com>

```
erfelipe@NoteBook-2: ~/Workspace/owlapi-fetch
+ owlapi git:(master) git push
Enumerating objects: 21, done.
Counting objects: 100% (21/21), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (11/11), 874 bytes | 174.00 KiB/s, done.
Total 11 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/erfelipe/owlRest.git
  80a82cf..65f8a4f  master -> master
+ owlapi git:(master)
Session Contents Restored on 5 Jun 2022 18:46
Last login: Sun Jun  5 18:46:30 on ttys000
+ owlapi git:(master) code .
+ owlapi git:(master) ls
Config-env.txt      Procfile           owlapi.iml        pom.xml            target
Exemplos.Formatos.Saida.txt  notas.txt          owlapi.war
+ owlapi git:(master) cd ..
+ Workspace ls
Ajax      Aula-Poo-Py      Django      NodeJS      aceview      crawlerByDomain  multistack      reactNative
AndroidStudio  Aula-Threads    Haskell    ReactCourse  aulaExpressJS  erfelipe.github.io owlapi      scraping-INPI
Aula-C      Aula1            Java-Poo   Servers      aulaFlask      html             owlapi-fetch  unifei
Aula-HTML-Form  AxiomValidator  JavaScript Temp          aulaHTML-CSS   imc              projetoAmanda  webcrawlerFX
Aula-Poo-Laz   Bcrypt          NextJS     WordCloud    aulaKubernetes messenger        projetoRI
+ Workspace ls owl*
owlapi:
Config-env.txt      Procfile           owlapi.iml        pom.xml            target
Exemplos.Formatos.Saida.txt  notas.txt          owlapi.war
owlapi-fetch:
codigo.js index.html
+ Workspace cd owlapi-fetch
+ owlapi-fetch git:(main) ls
codigo.js index.html
+ owlapi-fetch git:(main) code .
+ owlapi-fetch git:(main)
Session Contents Restored on 5 Jun 2022 22:20
Last login: Sun Jun  5 22:20:04 on ttys000
+ owlapi-fetch git:(main)
```



<https://www.erfelipe.com.br/git/>

Configuração do ambiente React Native

<https://reactnative.dev/docs/set-up-your-environment?os=windows&platform=android>

Configurações

- <https://reactnative.dev/docs/set-up-your-environment?os=windows&platform=android>
- Selecione o sistema operacional e siga a orientações
- Destaque para verificar:
 - Versões mínimas ou *específicas* dos softwares a serem instalados
 - Configurações das variáveis de ambiente
 - Configuração do **path** (no windows)

Expo

- Verificar as configurações em:
- <https://docs.expo.dev/get-started/set-up-your-environment/?mode=development-build&buildEnv=local>

Iniciando um projeto

npx

Para começar um projeto – CLI

- Entre na pasta (diretório) de estudos da disciplina.
- No prompt do terminal, usar:

`npx @react-native-community/cli@latest init primeiro`

- Este comando vai criar uma sub pasta (sub diretório) denominada "primeiro" na pasta atual.
- Vai demorar um tempo (considerável) para rodar o script e gerar a estrutura do projeto.
- Enquanto isso consulte:
 - <https://reactnative.dev/docs/getting-started-without-a-framework>

Para começar um projeto – Expo

- Entre na pasta (diretório) de estudos da disciplina.
- No prompt do terminal, usar:

npx create-expo-app@latest

- Este comando vai criar iniciar um script que perguntará o nome do projeto e criará uma pasta com o mesmo nome.
- Vai demorar um tempo (menor) para rodar o script e gerar a estrutura do projeto.
- Enquanto isso consulte:
 - <https://docs.expo.dev/get-started/create-a-project/>

Para processar um projeto

- Entre no Android Studio e inicie o **Simulador**:
 - More actions
 - Virtual Device Manager
 - Startar (ou criar) um simulador com a API 35 (neste momento de configuração)
- Após o processo de criação, para testar o projeto inicial, usa-se no PowerShell:
 - `cd pastaDoProjeto`
 - `npm run android`
- Expo
 - `npx expo start`

Para listar os emuladores Android disponíveis

- Para saber quais *emuladores* **Android** estão disponíveis, use no Terminal:
 - `emulator -list-avds`
 - `emulator -avd <nomeDoEmulador>`
- Para listar os dispositivos (celular, tablet) *conectados* e disponíveis para *deploy* (publicação) do projeto, use:
 - `adb devices`

Estrutura do modelo de desenvolvimento

Componentes

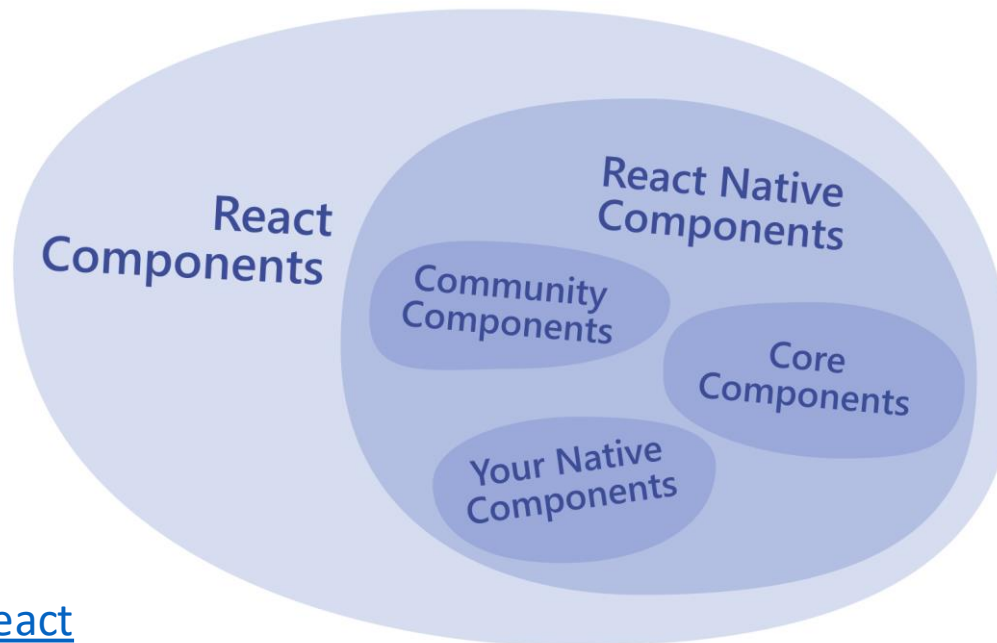
JSX

Props

State

Componentes

- React Native é baseado em outra biblioteca de desenvolvimento de aplicações (web): **React**, que por sua vez usa o **Node.js** como infra estrutura.
- O princípio de programação nestes ambientes está na concepção de **componentes**: estruturas menores (específicas) de software que quando combinadas, produzem resultados de interface e funcionalidade.
- JavaScript -> React Native -> Native Code (Android, iOS)



Funções

- Recurso para modularizar o código.
 - Facilitar manutenção;
 - Permite reuso;
- Sem RETORNO
 - Códigos que são executados e **não RESPONDEM** ao chamador;
 - Em algumas linguagens, denominadas VOID functions;
- Com RETORNO
 - Códigos que são executados e **RESPONDEM** com algum dado ao chamador;

Funções com retorno

```
#include <iostream>
```

```
using namespace std;
```

```
int soma(int a, int b) {  
    return a + b;  
}
```

```
int main() {  
    int resp = soma(10, 20);  
    cout << resp ;  
    return 0;  
}
```


Funções em JSX

- Uma função que RETORNA um resultado JSX, deve **retornar TAGs** aninhadas em **uma TAG** container;
- Essa função que retorna um conteúdo JSX é denominada Componente;

JSX

- Um componente funcional (baseado em funções) obedece algumas regras:
 - Uma função deve ser o **retorno default** do componente.
 - Deve sempre ser envolvida por uma **tag container**.

```
import { Text } from "react-native";
import { SafeAreaView } from 'react-native-safe-area-context';
export default function Teste() {
  return (
    <SafeAreaView>
      <Text>Teste</Text>
    </SafeAreaView>
  );
}
```

Estilo

style

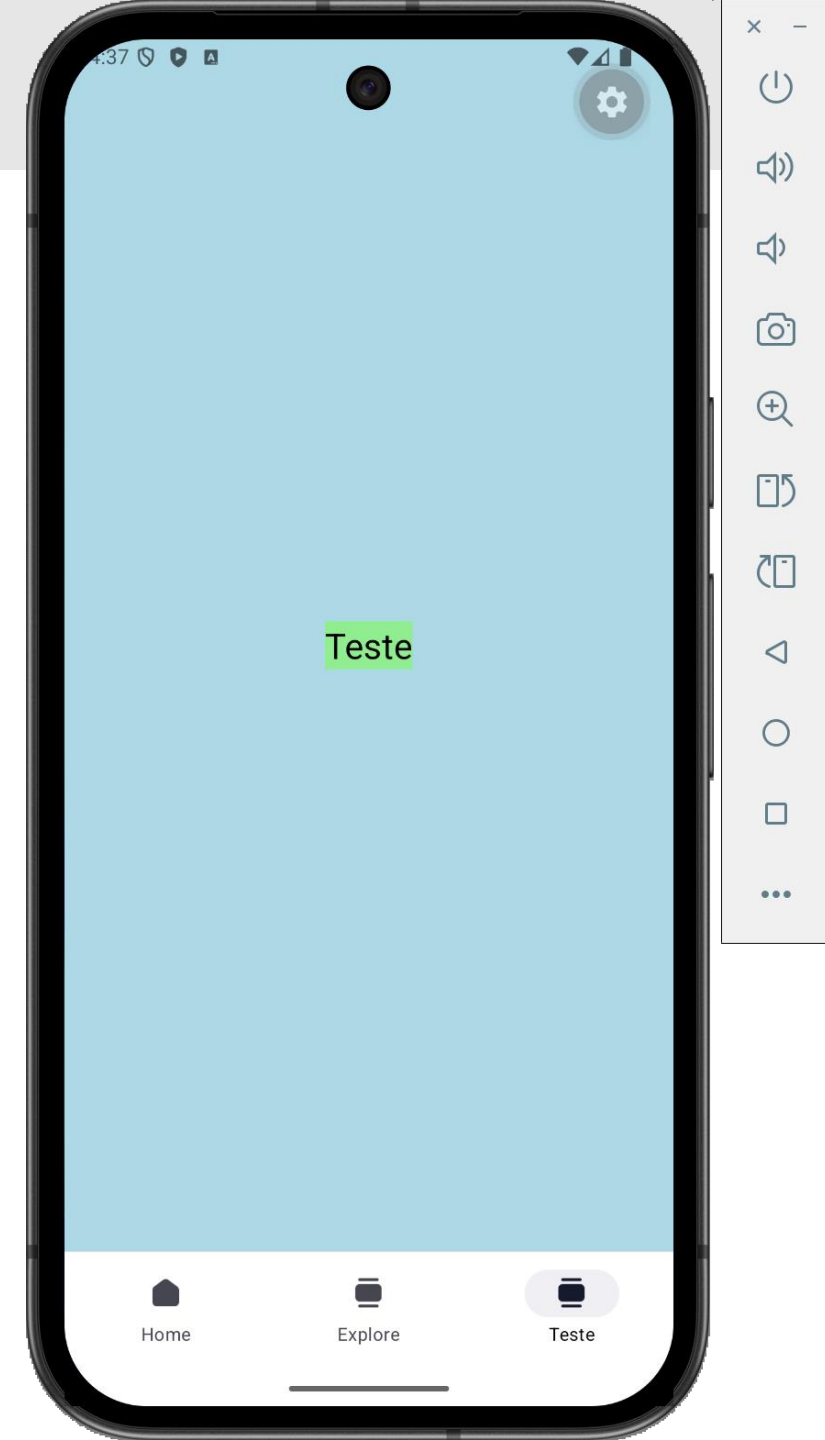
Modos de definição de estilo

- Diferente do que aprendemos anteriormente, onde o arquivo CSS era independente e servia a diversos arquivos HTML para formação, em um ambiente com a arquitetura baseada em componentes é indicado que a formatação esteja junto ao componente.
- Em react-native, usa-se a biblioteca **StyleSheet** para definir constantes CSS e aplicá-las ao componente.
- As configurações são definidas em formato de objeto {} .
- Importante: As constantes **CSS** são escritas em **sintaxe camelCase**.

Exemplo StyleSheet

```
import { StyleSheet, Text } from "react-native";
import { SafeAreaView } from 'react-native-safe-area-context';
export default function Teste() {
  return (
    <SafeAreaView style={estilos.container}>
      <Text style={estilos.importante}>Teste</Text>
    </SafeAreaView>
  );
}
```

```
const estilos = StyleSheet.create({
  container: {
    backgroundColor: 'lightblue',
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center'
  },
  importante: {
    backgroundColor: 'lightgreen',
    fontSize: 24
  }
});
```



Fechamento

React Native

- Usa uma sintaxe que mistura linguagens (JSX):
 - JavaScript
 - HTML
 - CSS
- Ao contrário de projetos convencionais, a organização do código em trechos menores (componentes) é o fundamento para a criação de aplicativos.
- Mesmo não produzindo código nativo (ao usar o Bridge), possui boa performance.

Analogia HTML e Components

HTML	React Native
div	View
img	Image
p, span	Text
ul/ol, li	ListView, child items

- Compra uma galinha que ela deixa o celular
- ...



eduardo.felipe@unifei.edu.br